

GLAM Protocol Audit Report

Table of Contents

1. Executive Summary

5

About GLAM

5

Audit Summary

5

Overall Security Posture

5

After Fix Review Summary

5

Before Fix Review Summary

5

Risk Profile

6

Launch Recommendations

7

Audit Scope

7

2. Assumptions and Considerations

8

Audit Assumptions

8

Centralization Risks

8

Trust Assumptions

8

3. Severity Definitions

9

Impact

9

Likelihood

9

Severity Classification Matrix

9

4. Findings

11

H01: Stale AUM after operations enables share dilution attacks

11

H02: Overpricing of external positions

13

M01: Admin cannot burn or force-transfer tokens from non-ATA accounts, allowing users to evade controls

15

M02: Compute units exhaustion in queue instructions

16

L01: Unstaked funds cannot be withdrawn from Kamino Farms

19

L02: Expired delegates can bypass authorization checks

20

L03: Admin can steal funds by maliciously configuring claimable fees

21

L04: Timelock can be bypassed allowing immediate state changes

22

L05: Rounding against the protocol in fulfill_subscription	23
L06: When setting new fees, they are applied retroactively	25
L07: Early exit in crystallize_fees_and_calc_new_shares means timestamps are not updated	26
L08: AUM calculation can be manipulated	27
L09: Usd-only oracle check blocks LSTs as base assets	29
L10: Fee crystallization before subscription allows users to receive more shares for the same deposit	30

5. Enhancement Opportunities 32

E01: Add a Check for Vault Type in CCTP Integration	32
E02: Hardcoded program ID used	33
E03: Redundant max operation on unsigned integer type	34
E04: Unused vec parameter in process_markets() adds unnecessary complexity	35
E05: Protocol fee configuration in GlobalConfig is not used during mint initialization	36
E06: Tokenized vaults can transfer SOL to any allowed address	37
E07: Kamino/Drift deposit and withdraw operations skip markets allowlist checks	38
E08: Strengthen Drift borrow guard	39
E09: Incorrect validation of stake pool account	40
E10: Eliminate dead code around underflow check	41
E11: Fail gracefully when trying to divide by zero by mistake	42
E12: Underwater position stalls pricing and AUM calculation due to problematic cast	43
E13: Vault fees documentation concern around burning	44
E14: Move crystallize_fees_and_calc_new_shares before fulfilling orders	45
E15: Ensure shares_decimals >= base_asset_decimals	46
E16: Add extra requirements before closing the GlamProtocol	47
E17: cpi_proxy could validate cpi_program the same way integration programs do	48
E18: Add threshold support against full amount delegate access	49
E19: Enforce Checks on extra_params for Each Action	50
E20: Lack of validation on destination accounts allows fund diversion by delegator	51
E21: Unsynced WSOL ATAs mispriced in validate_and_price_token_account	52

E22: Missing error handling for optional position valuation returns	54
E23: Inconsistent max age seconds configuration for aggregator oracles	55
E24: Output token account must be pre-created for Jupiter swaps	56
E25: Fee structure changes can be applied to pending requests when timelock is disabled	57
E26: Max-staleness oracle aggregation	58

About Us **59**

About Adevar Labs	59
Audit Methodology	59
1. Program Context and Architecture Analysis	59
2. Threat Modeling	59
3. In-depth Manual Security Review	60
4. Detailed Fix Review and Validation	60
Confidentiality Notice	60
Legal Disclaimer	61

Appendix A: LiteSVM Queue Stress Harness **62**

Findings at a Glance	62
Patch Context	62
Purpose	62
Test Setup	62
Tuning the Boundaries	62

Appendix B: AFL Fuzzing for Queue Subscriptions **64**

Patch Context	64
Findings at a Glance	64
Build & Run Workflow	64
Harness Overview	65
Invariants Enforced	65
Extending the Harness	66

Appendix C: Overpricing of external positions POC **67**

1. Executive Summary

About GLAM

GLAM is a protocol for programmable investment infrastructure on Solana. It provides composable primitives like vaults, mints, integrations, policies, and permissions to launch tokenized products and onchain strategies with precision and control.

Audit Summary

The following table summarizes the distribution of identified vulnerabilities by risk level:

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	2	2	0
Medium	2	2	0
Low	10	10	0

Enhancement opportunities span GLAM Protocol (15), GLAM Mint (6) and Integrations (5).

Overall Security Posture

GLAM’s architecture shows thoughtful separation between vault control and the various integrations, and the groundwork for a resilient platform is visibly in place. This section describes the security posture before and after the fix review. The fix review is the stage of the audit where Adevar Labs checked the fixes implemented by the GLAM development team for the issues found during the audit.

After Fix Review Summary

The post-remediation review confirms that the GLAM team fully addressed all 14 findings and delivered the majority of the requested enhancement work. The high, medium, and low severity items now fold in redesigned queue controls, tighter external pricing checks, and reinforced delegate guardrails. The codebase has no outstanding issues or caveats.

Before Fix Review Summary

During the initial assessment we noted valuation gaps in GLAM Mint that could enable share dilution, inflated external-position pricing, and request-queue liveness failures, alongside administrative guardrails that needed additional polish.

Risk Profile

Breakdown by product area:

I. GLAM Protocol

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	0	0	0
Low	5	5	0

II. GLAM Mint

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	2	2	0
Medium	2	2	0
Low	4	4	0

III. Integrations

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	0	0	0
Low	1	1	0

IV. GLAM Config

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	0	0	0
Low	0	0	0

V. GLAM Policies

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	0	0	0
Low	0	0	0

Launch Recommendations

With the remediation set merged, focus now shifts to validating the redesigned queue controls in production-like environments, institutionalizing the new guardrails across deployments, and sweeping the remaining integration quality-of-life improvements to keep momentum. We will include two appendices with tools to support this effort: (1) AFL-based fuzz harnesses that already target **subscribe** and can be extended to the rest of the queue flows, and (2) a LiteSVM stress test that exercises the queue to establish reliable operational bounds.

Audit Scope

- **Repository:** <https://github.com/glamsystems/glam>
- **Commit Hash:** `bafcfеab704126408fdd14c156a26901bdfb9dcb`
- **Programs in Scope:**
 - `anchor/programs/glam_mint`
 - `anchor/programs/glam_protocol`
 - `anchor/programs/glam_config`
 - `anchor/programs/glam_policies`
 - `anchor/programs/ext_kamino`
 - `anchor/programs/ext_drift`
 - `anchor/programs/ext_spl`
- **Exclusions:**
 - `anchor/programs/ext_marinade`
 - `anchor/programs/glam_protocol/src/instructions/stake.rs`
 - `anchor/programs/glam_mint/src/instructions/set_token_accounts_states.rs`
 - `anchor/programs/glam_mint/src/instructions/force_transfer_tokens.rs`
 - `anchor/programs/glam_mint/src/instructions/burn_tokens.rs`
 - `anchor/programs/glam_mint/src/instructions/mint_tokens.rs`

Afterward, the audit was extended to include:

- **Commit Hash:** `ca92a55e00494ce7d3c2e9a0fbb3739c63d77ac6`
- **Programs in Scope:** `anchor/programs/ext_cctp`
- **Files in Scope:**
 - `anchor/programs/glam_mint/src/instructions/set_token_accounts_states.rs`
 - `anchor/programs/glam_mint/src/instructions/force_transfer_tokens.rs`
 - `anchor/programs/glam_mint/src/instructions/burn_tokens.rs`
 - `anchor/programs/glam_mint/src/instructions/mint_tokens.rs`

All remediation pull requests have landed on **main**, which currently tips at commit `09be458be9f296ff20e03cad75525a8d6807071d`.

2. Assumptions and Considerations

Audit Assumptions

I. Deployment Integrity: Programs are deployed from the reviewed source baseline, and upgrade authorities are exercised solely through the documented governance process.

II. Third-Party Integrations: Integrated protocols such as Drift, Kamino, Marinade, SPL Stake Pool, Switchboard, and Pyth are assumed to operate as specified; vulnerabilities in those systems fall outside this review.

III. Oracle Availability: USD-denominated oracle feeds remain timely and manipulation-resistant; extreme deviations or outages are treated as environmental risk.

Centralization Risks

- GLAM owner is limited to registry maintenance: they can rotate the global referrer, curate asset metadata, and adjust the protocol fee basis points that flow out of vaults.
- Each vault's owner key governs the vault itself and retains full-control capabilities. Owner-signed calls are required to toggle integrations, manage delegate ACLs, or change timelock settings.
- All deployed programs remain upgradeable, so users rely on the upgrade authority to steward releases responsibly.

Trust Assumptions

- Vault depositors ultimately trust both the GLAM owner and the specific vault owner to act in their best interests; depositing effectively hands strategy discretion to those parties.
- Delegates must operate in the vault's best interest, since their permissions lack on-chain thresholds and could otherwise be abused before the vault owner intervenes.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements—typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product’s reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue, particularly on **project funds, user funds**, and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

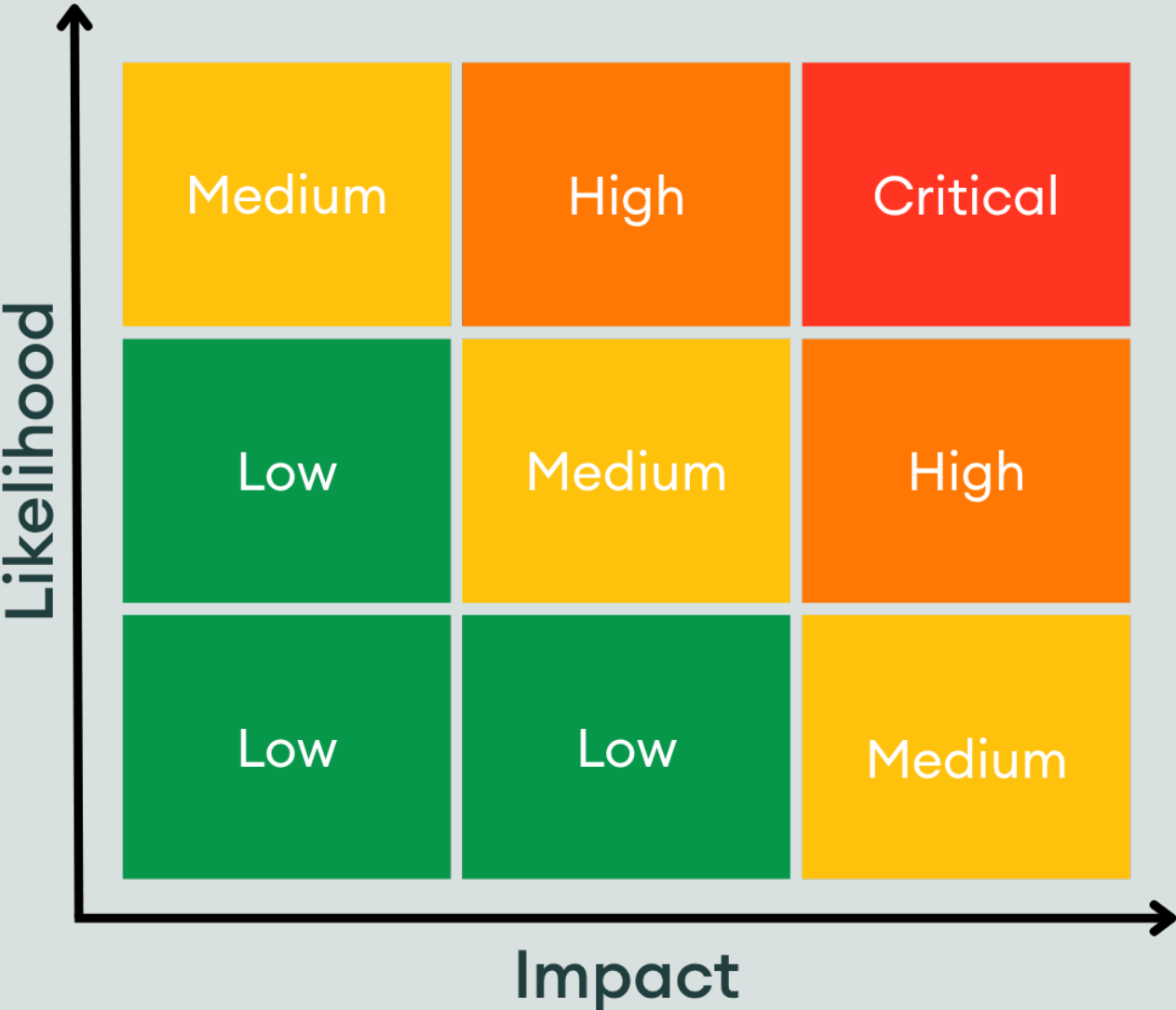
- **High Likelihood:** The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood:** This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood:** Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Moderate impact and/or discoverability
- **Low:** Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

H01: Stale AUM after operations enables share dilution attacks

Status:

Resolved

Impact:



Likelihood:



Severity:

High

Location: https://github.com/glamsystems/glam/blob/bafcf704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_mint/src/state/aum.rs#L16-L20

>_aum.rs

RUST

```
14:    /// Prices vault tokens and external positions in base asset
15:    fn aum(&self, slot: u64) -> Result<i128> {
16:        let priced_protocols = self
17:            .priced_protocols
18:            .iter()
19:            .filter(|p| p.last_updated_slot >= slot)
20:            .collect::<Vec<_>>();
21:
22:        // Vault tokens must be priced (special entry with integration_program == Glam
        Protocol::id() and protocol_bitflag == 0)
```

Description:

The protocol tracks Assets Under Management (AUM) in **StateAccount.priced_protocols**, a vector of **PricedProtocol** structs. Each struct stores an **amount** field with the priced value of assets and a **last_updated_slot** that records when pricing occurred. Pricing instructions such as **price_vault_tokens()**, **price_drift_users()**, and similar flows update **priced_protocols** via the CPI helper **update_priced_protocol()**. The **aum()** function then calculates total AUM by summing the amounts in that vector.

The bug stems from how cached AUM is reused after state-changing operations. Instructions like **fulfill()**, **subscribe()**, **crystallize_fees()**, and **claim_fees()** rely on the cached values stored in **priced_protocols** to derive share ratios. Those instructions also mutate the vault state (minting or burning shares and moving assets) but never invalidate or refresh **priced_protocols**.

This creates a window in which **priced_protocols** holds stale information that no longer reflects on-chain balances. Subsequent transactions in the same slot read that stale data via **aum()** and compute incorrect share amounts.

Scenario within one block:

1. **price_all_protocols** executes. AUM = 1,000 USD, shares = 1,000, ratio = 1:1.
2. **fulfill()** processes redemption requests for 800 shares, burns them, and transfers 800 USD out. **StateAccount.priced_protocols** still reports AUM = 1,000 USD while supply is now 200 shares.
3. A user calls **subscribe()** with 100 USD. The function reads AUM = 1,000 USD and supply = 200

shares, calculates a 4:1 ratio, and mints only 25 shares instead of the correct 100.

Early redeemers gain at the expense of later participants by leveraging stale AUM values. The impact grows with larger redemptions occurring immediately before new subscriptions.

Recommendation:

Reset or invalidate the cached **priced_protocols** entries immediately after consuming the AUM value so that any subsequent operation forces a fresh price update.

Developer Response:

Fixed by resetting **priced_protocols** at the end of **subscribe**, **fulfill**, and **claim_fees**.
Change: <https://github.com/glamsystems/glam/pull/828>.

H02: Overpricing of external positions

Status:

Resolved

Impact:

Low Medium High

Likelihood:

Low Medium High

Severity:

High

Location: https://github.com/glamsystems/glam/blob/bafcfceab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_mint/src/state/aum.rs#L31-L44

```

>_aum.rs RUST
29:         );
30:
31:         // All external positions must be priced. self.external_positions is a vec but
it guarantees no dupes:
32:         // set(priced_positions) == set(self.external_positions)
33:         let priced_positions: HashSet<_> = priced_protocols
34:             .iter()
35:             .flat_map(|p| p.positions.iter().copied())
36:             .collect();
37:         if priced_positions.len() != self.external_positions.len() {
38:             msg!(
39:                 "priced_positions.len: {} != state.external_positions.len: {}",
40:                 priced_positions.len(),
41:                 self.external_positions.len()
42:             );
43:             return err!(GlamError::ExternalPositionsNotPriced);
44:         }
45:         for external_position in &self.external_positions {
46:             if !priced_positions.contains(external_position) {

```

Description:

The current implementation converts **priced_positions** into a **HashSet** before comparing it with **external_positions**. **external_positions** is guaranteed to be duplicate-free, but **priced_positions** can contain duplicates. If **priced_positions** is **{A, A, A, A}** and **external_positions** is **{A}**, the set comparison succeeds because both sets evaluate to **{A}**, and the AUM calculation proceeds.

Some pricing endpoints, such as **price_drift_users**, check for duplicates:

```

RUST
require_eq!(
    priced_protocol.positions.len(),
    user_stats.number_of_sub_accounts as usize,
    GlamError::MissingAccount,
);

```

Others, including **price_kamino_vault_shares**, do not enforce uniqueness:

RUST

```
if !tracked.contains(shares_ata_account_info.key) {  
    msg!(  
        "Skipping untracked shares ata: {}",  
        shares_ata_account_info.key()  
    );  
    continue;  
}
```

An endpoint that prices the same position repeatedly can inflate AUM.

Proof of Concept:

See Appendix 3.

Recommendation:

Either compare **priced_positions** directly as a vector against **external_positions** or prevent pricing endpoints from emitting duplicates. Ideally, implement both defenses.

Developer Response:

Fixed by not allowing duplicated positions.

Change: <https://github.com/glamsystems/glam/pull/781>.

M01: Admin cannot burn or force-transfer tokens from non-ATA accounts, allowing users to evade controls

Status:

Resolved

Impact:

Low Medium High

Likelihood:

Low Medium High

Severity:

Medium

Location: https://github.com/glamsystems/glam/blob/ca92a55e00494ce7d3c2e9a0fbb3739c63d77ac6/anchor/programs/glam_mint/src/instructions/burn_tokens.rs#L55-L61

>_ burn_tokens.rs

RUST

```
53:     pub glam_mint: InterfaceAccount<'info, Mint>,
54:
55:     #[account(
56:         mut,
57:         associated_token::mint = glam_mint,
58:         associated_token::authority = from,
59:         associated_token::token_program = token_2022_program
60:     )]
61:     pub from_ata: Box<InterfaceAccount<'info, TokenAccount>>,
62:
63:     /// CHECK: any address owned by system program
```

Description:

burn_tokens() and **force_transfer_tokens()** restrict **from_token** to associated token accounts (ATAs). Admins should be able to burn or transfer from any SPL token account issued by the mint. The current constraint lets malicious users move tokens into non-ATA accounts, causing admin operations to fail validation and effectively bypassing controls.

Recommendation:

Replace the ATA-specific constraint with standard token-account validation in both instructions.

Developer Response:

Fixed by making **burn** and **force_transfer** work from any token account.
Change: <https://github.com/glamsystems/glam/issues/821>.

M02: Compute units exhaustion in queue instructions

Status:

Resolved

Impact:

Low Medium High

Likelihood:

Low Medium High

Severity:

Medium

Location: https://github.com/glamsystems/glam/blob/bafcf0eab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_mint/src/state/request_queue.rs#L134-L137

>_request_queue.rs

RUST

```
132:         let mut agg_pa_shares_manager_redemption_fee: u128 = 0;
133:
134:         for request in self
135:             .data
136:             .iter_mut()
137:             .filter(|r| match r.request_type { RequestType::Subscription => r.created_at <= subscription_cutoff, RequestType::Redemption => r.created_at <= redemption_cutoff } && r.is_pending_fulfillment())
138:         {
139:             match request.request_type {
```

Description:

The **fulfill** method processes the entire eligible subset of the **RequestQueue** in a single instruction. The queue is only bounded by the Solana account size limit (10 MiB). Even with a one-request-per-user rule, a large user base can still produce a queue with tens of thousands of entries. When liquidity is available, **RequestQueue::fulfill** tries to iterate through every eligible request, exhausting compute units, especially because fulfilled requests remain in the queue until users claim them. Once the queue grows large, the whole flow can become stuck.

This $O(n)$ iteration pattern also appears in other queue helpers such as **.search** and **.retain**.

Let's run the numbers:

- The queue is capped by the 10 MiB account size: $10\ 1024\ 1024 = 10,485,760$ bytes.
- Each **PendingRequest** entry occupies 72 bytes.
- Therefore, the queue can hold up to roughly 145,635 requests.
- Solana's per-transaction compute limit is 1.4 million CU.

Looking at the **fulfill** instruction:

- It loops over the entire queue, so in theory it must handle all 145,635 entries.
- Fully processing a request (roughly 10-20 emulated big-int ops) costs about 2,000-5,000 CU when the iteration matches.
- Assuming a 50% match rate, the blended cost is around 1,000-2,000 CU per iteration.
- That means the loop exceeds 1.4 million CU after roughly 700-1,400 iterations.
- If every entry matches the filter, the ceiling drops to 300-700 iterations.
- Beyond those thresholds, the transaction fails with no state changes.

In the worst case (queue full, every request matching) the loop would require roughly 284 million to 711 million CU to complete, far beyond the 1.4 million CU limit.

subscribe and **redeem** show similar behavior: both scan the entire vector to enforce the “one

active request per user” constraint. As the queue grows, these instructions also hit compute limits and start failing, leaving the queue wedged.

This analysis should be confirmed with a LiteSVM proof of concept to validate the rough benchmarks and assumptions.

Recommendation:

The mitigation depends on expected load. If request volume stays low and a trusted delegate routinely drains the queue, a simple hard cap on queue length might suffice (acknowledging that large queues still incur high transaction costs). Otherwise, the queue should be reimplemented on top of a data structure designed for sublinear operations.

The current **Vec**-backed queue offers:

- $O(n)$ search.
- $O(1)$ insertion in theory, but effectively $O(n)$ because every insert performs a search.
- $O(n)$ deletion (**redeem**) due to two searches plus a left shift.

Alternatives considered:

1. Store each **PendingRequest** as a PDA. This preserves on-chain state but breaks FIFO behavior.
2. Use per-request PDAs with a doubly linked list. That yields $O(1)$ operations but requires passing every PDA into **fulfill**, quickly hitting Solana's 1,232-byte transaction limit. Fulfillment would need batching and an off-chain account-selection helper, which is unattractive.
3. Combine per-user PDAs with bucketed queue PDAs, for example:

RUST

```
#[account(zero_copy)]
pub struct RequestQueue {
    pub glam_state: Pubkey,
    pub glam_mint: Pubkey,
    pub subscription_paused: bool,
    pub redemption_paused: bool,
    pub current_bucket_id: u64, // Next for adds
    pub next_process_bucket_id: u64, // Next for fulfill
    pub bucket_size_limit: usize, // e.g., 200 entries
}

#[account(zero_copy)]
pub struct Bucket {
    pub queue: Pubkey,
    pub id: u64,
    pub len: usize,
    pub data: [PendingRequest; 200], // Fixed array; zero-copy access
}

#[account]
pub struct UserPointer {
    pub bucket_id: u64,
    pub index: usize, // Position in bucket.data
}

// PendingRequest unchanged
```

This layout keeps search, insert, and delete at $O(1)$ via the **UserPointer**, while **fulfill** only needs the PDAs for buckets that actually contain requests. It still introduces complexity, but it allows batching without unbounded iteration and is a workable starting point for redesign.

Developer Response:

Fixed by:

- Reworking the structure to implement a zero-copy-like request queue.
- Making **claim** permissionless.
- Adding an optional **limit** argument to **fulfill**.

Change: <https://github.com/glamsystems/glam/pull/758>.

L01: Unstaked funds cannot be withdrawn from Kamino Farms

Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: https://github.com/glamsystems/glam/blob/bafcf9ab704126408fdd14c156a26901bdfb9dcb/anchor/programs/ext_kamino/src/instructions/farms_unstake.rs#L29-L31

>_ farms_unstake.rs

RUST

```
27:         _stake_shares_scaled: stake_shares_scaled,
28:     };
29:     let mut data = Vec::with_capacity(256);
30:     data.extend_from_slice(kamino_farm::instruction::Unstake::DISCRIMINATOR);
31:     AnchorSerialize::serialize(&ix, &mut data)?;
32:
33:     let mut remaining_accounts = vec![]
```

Description:

farms_unstake() CPI-calls Kamino Farms' **unstake()** instruction, which only marks the position as unstaked and leaves funds in Kamino. Completing the withdrawal requires a follow-up CPI to **withdraw_unstaked_deposits()**, but the current implementation never performs that call, so funds remain stranded.

Recommendation:

Add a helper that invokes Kamino Farms' **withdraw_unstaked_deposits()** after unstaking so the tokens return to **glam_vault**.

Developer Response:

Fixed by adding support for farms instructions **initializeUser** and **withdrawUnstakedDeposits**. Change: <https://github.com/glamsystems/glam/pull/826>.

L02: Expired delegates can bypass authorization checks

Status:

Resolved

Impact:



Low

Medium

High

Likelihood:



Low

Medium

High

Severity:

Low

Location: https://github.com/glamsystems/glam/blob/bafcfeab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/state/auth.rs#L153-L154

> _auth.rs

RUST

```
151: }  
152:  
153: pub fn check_owner_or_delegate(state: &StateAccount, signer: &Pubkey) -> Result<()> {  
154:     if state.owner == *signer {  
155:         return Ok(());  
156:     }
```

Description:

`check_owner_or_delegate()` iterates over `delegate_acls` and returns success as soon as it finds a matching `pubkey`. The function never verifies the `expires_at` timestamp, so delegates remain authorized even after their permissions expire.

Recommendation:

Validate `expires_at` before treating a delegate as authorized, consistent with the checks already implemented in `check_access_any()`.

Developer Response:

Fixed by adding an expiration check to `check_owner_or_delegate`.
Change: <https://github.com/glamsystems/glam/pull/782>.

L03: Admin can steal funds by maliciously configuring claimable fees

Status:

Resolved

Impact:

Low

Medium

High

Likelihood:

Low

Medium

High

Severity:

Low

Location: https://github.com/glamsystems/glam/blob/bafcf9eb704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/state/accounts.rs#L393-L395

> _accounts.rs

RUST

```
391:         for update in self.mint_update_next().clone() {
392:             // Only NotifyAndSettle and FeeStructure can be changed by user
393:             if update.name == EngineFieldName::NotifyAndSettle
394:                 || update.name == EngineFieldName::FeeStructure
395:             {
396:                 self.mint_params_mut()
397:                     .set(update.name, update.value.clone());
```

Description:

`update_mint_params()` allows the administrator to update parameters immediately when `timelock_duration` is 0. In that mode the function does not validate which parameter types can be updated, so an admin can modify **ClaimableFees** in place and redirect most or all fees to themselves.

When `timelock_duration > 0`, updates are staged and only **NotifyAndSettle** and **FeeStructure** are permitted when the timelock expires. The zero-timelock path lacks equivalent validation.

Recommendation:

1. Let `initialize_mint` call a helper instruction that allows the mint program to update any parameter (`update_mint_params_by_mint_authority`).
2. Restrict the zero-timelock case to only allow **NotifyAndSettle** and **FeeStructure**.

Developer Response:

Fixed by preventing updates on unintended internal states.
Change: <https://github.com/glamsystems/glam/issues/770> and <https://github.com/glamsystems/glam/pull/796>.

L04: Timelock can be bypassed allowing immediate state changes

Status:

Resolved

Impact:

Low

Medium

High

Likelihood:

Low

Medium

High

Severity:

Low

Location: https://github.com/glamsystems/glam/blob/bafcf9ab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/state/accounts.rs#L405-L407

> _accounts.rs

RUST

```
403:     }  
404:  
405:     pub fn is_timelock_expired(&self) -> bool {  
406:         self.timelock_expires_at <= now::().unwrap()  
407:     }  
408:  
409:     /// Clears staged updates to state & mint params and resets timelock expiration
```

Description:

`is_timelock_expired()` returns `true` when `StateAccount.timelock_expires_at` is `0`. This lets callers bypass the timelock:

1. Call `update_state_apply_timelock()`, which applies a change and resets `timelock_expires_at` to `0`.
2. Invoke an instruction such as `enable_disable_protocols_handler()`, `grant_revoke_delegate_permissions()`, `set_protocol_policy()`, `set_jupiter_swap_policy()`, or `set_system_transfer_policy()`. These stage updates in `StateAccount.params[2]` but do not update `timelock_expires_at`, leaving it at `0`.
3. Call `update_state_apply_timelock()` again. Because `timelock_expires_at` is `0`, the check succeeds immediately and the staged changes become active with no delay.

Recommendation:

Whenever the timelock feature is enabled and an instruction stages changes, call `state_account.extend_timelock_expiration()` so the new timelock is enforced.

Developer Response:

Fixed according to the recommendation.

Change: <https://github.com/glamsystems/glam/pull/788>.

L05: Rounding against the protocol in fulfill_subscription

Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: https://github.com/glamsystems/glam/blob/bafcf9ab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_mint/src/state/request_queue.rs#L54-L57

>_request_queue.rs

RUST

```
52:         fee_structure.manager.subscription_fee_bps,
53:     )?;
54:     let total_fee_shares = pa_shares_vault_fee
55:         .safe_add(pa_shares_manager_fee)?
56:         .safe_div(PRECISION_ADJUSTMENT.cast())?
57:         .cast::<u64>()?;
58:     let shares_after_fees = shares_before_fees.safe_sub(total_fee_shares)?;
59:
```

Description:

total_fee_shares is computed using floor division. The user receives **shares_after_fees = shares_before_fees - total_fee_shares**, so rounding down the fees effectively rounds the user's shares up. Protocols typically round fees up to stay solvent; rounding down slowly bleeds value and can break invariants.

Proof of Concept:

Starting from the already present test **test_fulfill_subscription_only**. This test basically subscribes **1_000_000** to the queue. The fee structure is: 1% vault sub, 0.5% vault red, 0.5% manager sub, 0.25% manager red.

As expected, the test passes those checks:

RUST

```
// Should have positive shares and base asset amounts for subscription
assert_eq!(agg_shares_amount, 990_000); // 1% vault sub fee
assert_eq!(agg_base_asset_amount, 1_000_000);
assert_eq!(agg_pa_shares_vault_subscription_fee, 10_000_000_000_000);
assert_eq!(agg_pa_shares_manager_subscription_fee, 5_000_000_000_000);
assert_eq!(agg_pa_shares_vault_redemption_fee, 0);
assert_eq!(agg_pa_shares_manager_redemption_fee, 0);

// Request should be fulfilled
assert_eq!(pending_requests.data[0].fulfilled_at, now);
assert_eq!(pending_requests.data[0].outgoing, 985_000);
```

But, when changing the subscription amount to **1_000_001** (one more wei from what was tested initially), the test passes with this values:

RUST

```
// Should have positive shares and base asset amounts for subscription
assert_eq!(agg_shares_amount, 990_001); // 1% vault sub fee
assert_eq!(agg_base_asset_amount, 1_000_001);
assert_eq!(agg_pa_shares_vault_subscription_fee, 10_000_010_000_000);
assert_eq!(agg_pa_shares_manager_subscription_fee, 5_000_005_000_000);
assert_eq!(agg_pa_shares_vault_redemption_fee, 0);
assert_eq!(agg_pa_shares_manager_redemption_fee, 0);

// Request should be fulfilled
assert_eq!(pending_requests.data[0].fulfilled_at, now);
assert_eq!(pending_requests.data[0].outgoing, 985_001);
```

This means the user gets **985_001** shares, **shares_vault_subscription_fee** (without precision adjustment) is **10_000,01** and **manager_subscription_fee** (without precision adjustment) is **5_000,005**. In total, shares that are issued will be **985_001 + 10_000,01 + 5_000,005 = 1_000_001,015**. Notice the **,015** at the end. This does not happen of course in the program because there are integers, not floats, but still, the user should (theoretically) get **1_000_001 - 10_000,01 - 5_000,005 = 985_000,005** (so **985_000** casting to an integer). Instead, because the sum of the fees are rounded down, he gets that 1 extra wei difference (**985_001** instead of **985_000**).

Recommendation:

Round fees up when computing **total_fee_shares**:

RUST

```
let total_fee_shares = pa_shares_vault_fee
    .safe_add(pa_shares_manager_fee)?
    .safe_add(PRECISION_ADJUSTMENT as u128 - 1)? // for ceiling division
    .safe_div(PRECISION_ADJUSTMENT.cast())?
    .cast::<u64>()?;
```

The same adjustment is needed in **fulfill_redemption**. With ceiling math in place, the workaround **std::cmp::min(to_burn, ctx.accounts.escrow_mint_ata.amount)** in **claim_fees.rs** should become unnecessary.

Developer Response:

Fixed according to the recommendation.

Change: <https://github.com/glamsystems/glam/pull/780>.

L06: When setting new fees, they are applied retroactively

Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: https://github.com/glamsystems/glam/blob/bafcf0eab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_mint/src/instructions/set_protocol_fees.rs#L33-L40

> **set_protocol_fees.rs**

RUST

```
31:     check_owner_or_delegate(&ctx.accounts.glam_state, ctx.accounts.signer.key)?;  
32:  
33:     let fee_structure = ctx.accounts.glam_state.get_fee_structure().unwrap();  
34:     let updated_fee_structure = FeeStructure {  
35:         protocol: ProtocolFees {  
36:             base_fee_bps,  
37:             flow_fee_bps,  
38:         },  
39:         ..fee_structure.clone()  
40:     };  
41:  
42:     glam_protocol::cpi::update_mint_params_by_mint_authority(
```

Description:

set_protocol_fees updates the fee structure without crystallizing outstanding fees first, effectively applying the new rates retroactively.

Recommendation:

Crystallize fees before updating the protocol fee configuration.

Developer Response:

Fixed checking that protocol fees were crystallized before updating.

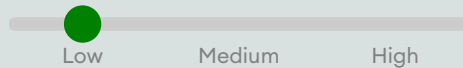
Change: <https://github.com/glamsystems/glam/pull/799>.

L07: Early exit in `crystallize_fees_and_calc_new_shares` means timestamps are not updated

Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: https://github.com/glamsystems/glam/blob/bafcfcab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/state/flow.rs#L380-L382

>_flow.rs

RUST

```
378:     shares_decimals: u8,  
379: ) -> Result<u64> {  
380:     if aum <= 0 {  
381:         return Ok(0);  
382:     }  
383:  
384:     let FeeParams {
```

Description:

`crystallize_fees_and_calc_new_shares` returns early when `aum <= 0`. That shortcut skips updates to `fee_structure` timestamps and `last_aum`, breaking performance-fee calculations that depend on `last_performance_fee_crystallized`.

Recommendation:

Remove the early return. Later logic already handles `aum <= 0` while still updating timestamps.

Developer Response:

Fixed according to the recommendation.

Change: <https://github.com/glamsystems/glam/pull/759>.

L08: AUM calculation can be manipulated

Status:

Resolved

Impact:

Low Medium High

Likelihood:

Low Medium High

Severity:

Low

Location: https://github.com/glamsystems/glam/blob/bafcfceab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_mint/src/state/aum.rs#L45-L50

>_aum.rs

RUST

```
43:         return err!(GlamError::ExternalPositionsNotPriced);
44:     }
45:     for external_position in &self.external_positions {
46:         if !priced_positions.contains(external_position) {
47:             msg!("External position not priced: {}", external_position);
48:             return err!(GlamError::ExternalPositionsNotPriced);
49:         }
50:     }
51:
52:     let aum = priced_protocols
```

Description:

The AUM calculation is vulnerable to manipulation when external position pubkeys are reused (for example, in Drift integrations). If a high-value position is deleted and a low-value position is recreated at the same pubkey within the same slot, subsequent pricing can reuse stale **PricedProtocol** data and inflate subscriptions.

Consider the following scenario at slot X, where all relevant **priced_protocols** have **last_updated_slot >= X** and pass initial filters. The calculation runs through these checks:

Check 1

```
require!(
    vault_tokens_priced.is_some(),
    GlamError::VaultTokensNotPriced
);
```

RUST

This passes in the scenario.

Check 2

```
if priced_positions.len() != self.external_positions.len() {
    msg!(
        "priced_positions.len: {} != state.external_positions.len: {}",
        priced_positions.len(),

```

RUST

... continued

RUST

```
        self.external_positions.len()
    );
    return err!(GlamError::ExternalPositionsNotPriced);
}
```

This also passes.

Check 3

RUST

```
for external_position in &self.external_positions {
    if !priced_positions.contains(external_position) {
        msg!("External position not priced: {}", external_position);
        return err!(GlamError::ExternalPositionsNotPriced);
    }
}
```

The problem is that this check only confirms the pubkey exists; it never verifies that the underlying position is unchanged. Reusing a pubkey lets the protocol rely on outdated pricing from earlier in the slot and overstate AUM.

Attack steps within the same slot:

1. Delete the valuable position with **delete_user** (Drift integration). The CPI closes the account, reduces the on-chain value, and removes the pubkey from **external_positions**:

RUST

```
vec![ExtraParams {
    action_type: ActionType::DeleteExternalAccount,
    pubkey: ctx.accounts.user.key(),
    amount: None,
}],
```

2. Recreate a low-value position with **initialize_user** using the same **sub_account_id**, which recreates the PDA at the same pubkey and adds it back to **external_positions**:

RUST

```
vec![ExtraParams {
    action_type: ActionType::AddExternalAccount,
    pubkey: ctx.accounts.user.key(),
    amount: None,
}],
```

PricedProtocols now contains stale, high-value data. A subsequent subscription in slot X calls **aum()**, passes every check (pubkeys match), and mints shares against the outdated valuation.

Recommendation:

Add a state flag alongside **priced_protocols** and invalidate it whenever an external position is added or removed.

Developer Response:

Fixed by implementing a reset on **priced_protocols** whenever **cpi_proxy** is called.
Change: <https://github.com/glamsystems/glam/issues/748>.

L09: Usd-only oracle check blocks LSTs as base assets

Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: https://github.com/glamsystems/glam/blob/bafcf9ab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/state/assets.rs#L31-L35

```
> _assets.rs RUST
29:         GlamError::InvalidPricingOracle
30:     );
31:     require_eq!(
32:         base_asset_usd_price.denom,
33:         PriceDenom::USD,
34:         GlamError::InvalidPricingOracle
35:     );
36:
37:     // convert lamports to scaled usd
```

Description:

Inside **PricedAssetInfo::base_asset_amount** the vault insists on a USD-denominated oracle for the base asset (**require_eq!(base_asset_usd_price.denom, PriceDenom::USD, ...)**). Liquid staking tokens report prices in SOL terms, so any vault that chooses an LST as its base asset immediately hits **InvalidPricingOracle** and cannot function. This removes a common, intended configuration from the product.

Recommendation:

Keep the existing **require_eq!(sol_usd_price.denom, PriceDenom::USD, ...)** so the SOL reference feed must be USD-denominated. Replace the second **require_eq!** with a **match** on **base_asset_usd_price.denom**: if it's **PriceDenom::USD**, reuse the current USD -> base-asset conversion; if it's **PriceDenom::SOL**, first turn the aggregated USD value into SOL lamports via the SOL oracle and then feed those lamports through the LST oracle to obtain the base-asset amount.

Developer Response:

Fixed according to the recommendation.

Change: <https://github.com/glamsystems/glam/issues/836>.

L10: Fee crystallization before subscription allows users to receive more shares for the same deposit

Status:

Resolved

Impact:



Likelihood:



Severity:

Low

Location: https://github.com/glamsystems/glam/blob/bafcf0eab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_mint/src/instructions/subscribe.rs#L192-L201

>_subscribe.rs

RUST

```
190:
191:    // Crystallize fees and mint fee shares to escrow
192:    let misc_fee_shares = crystallize_fees_and_calc_new_shares(
193:        fee_structure,
194:        fee_params,
195:        claimable_fees,
196:        aum,
197:        base_asset_decimals,
198:        Clock::get()?.unix_timestamp,
199:        shares_supply,
200:        shares_mint_decimals,
201:    )?;
202:    let manager_subscription_fee_shares = shares_to_user_and_escrow.safe_sub(shares_to
    _user)?;
203:    let escrow_new_fee_shares = manager_subscription_fee_shares.safe_add(misc_fee_shar
    es)?;
```

Description:

The **subscribe()** function calculates Net Asset Value (NAV) at the beginning to determine share allocation. However, the permissionless **crystallize_fees()** function can be called immediately before **subscribe()**, creating an exploitable inconsistency.

NAV is calculated as: **aum decimals_adjustment PRECISION_ADJUSTMENT / shares_supply**, making it inversely proportional to total share supply.

The attack sequence:

1. **crystallize_fees()** is permissionless and can be called by anyone. It calculates accrued fees and mints new shares to escrow, increasing total supply. Increased supply causes NAV to decrease.
2. **subscribe()** then calculates NAV using this inflated supply. Lower NAV means the user receives more shares for the same deposit amount.

While **subscribe()** internally calls **crystallize_fees_and_calc_new_shares()** later, this happens after the initial NAV calculation has already determined base share allocation.

Users can take advantage of this behaviour to call **crystallize_fees()** before **subscribe()**, systematically receiving more shares than intended for the same deposit. The behaviour is internally inconsistent, and it does let proactive users get a better price, even though the

absolute gain is limited (they're really just avoiding stale-fee dilution).

Recommendation:

Execute `crystallize_fees_and_calc_new_shares()` before calculating NAV for share allocation. Use the post-crystallization state (with fee shares already minted) to calculate NAV, ensuring consistency regardless of external `crystallize_fees()` calls. Apply this pattern to all instructions performing NAV-based share calculations, which are `subscribe()` and `fulfill()` instructions.

Developer Response:

Fixed according to the recommendation.

Change: <https://github.com/glamsystems/glam/issues/837>.

5. Enhancement Opportunities

E01: Add a Check for Vault Type in CCTP Integration

Location: https://github.com/glamsystems/glam/blob/ca92a55e00494ce7d3c2e9a0fbb3739c63d77ac6/anchor/programs/ext_cctp/src/instructions/deposit_for_burn.rs#L23

```
> _deposit_for_burn.rs RUST
21:
22: #[integration_authority_signer_seeds]
23: pub fn deposit_for_burn<'c: 'info, 'info>(
24:     ctx: Context<'_, '_, 'c, 'info, DepositForBurn<'info>>,
25:     params: DepositForBurnParams,
```

Description:

The current CCTP deposit-and-burn instruction is designed for Vault accounts only, but the code never enforces that requirement.

Potential Benefit:

Clarifies the intended usage and prevents unintended account types from interacting with the flow.

Recommendation:

Add an explicit Vault-type assertion to the CCTP integration before processing the instruction.

E02: Hardcoded program ID used

Location: https://github.com/glamsystems/glam/blob/bafcf9eb704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/instructions/mint.rs#L85-L86

```
> _mint.rs RUST
83:         mut,
84:         seeds = [SEED_MINT.as_bytes(), &[0], glam_state.key().as_ref()],
85:         seeds::program = pubkey!("GM1NtvvnSXUptTrMCqbogAdZJydZSNv98DoU5AZVLmGh"),
86:         bump,
87:     )]
88:     pub glam_mint: InterfaceAccount<'info, Mint>,
```

Description:

`TokenTransferCheckedByMintAuthority` hardcodes `pubkey!("GM1NtvvnSXUptTrMCqbogAdZJydZSNv98DoU5AZVLmGh")` to validate the `glam_mint` program, while `UpdateStateByMintAuthority` in the same module correctly references the shared constant `GLAM_MINT_PROGRAM_ID`. Mixing the two styles reduces readability and risks drift if the program ID changes.

Potential Benefit:

Improves consistency and avoids maintenance drift.

Recommendation:

Replace the hardcoded literal with the `GLAM_MINT_PROGRAM_ID` constant:

```
- seeds::program = pubkey!("GM1NtvvnSXUptTrMCqbogAdZJydZSNv98DoU5AZVLmGh"),
+ seeds::program = GLAM_MINT_PROGRAM_ID,
```

DIFF

E03: Redundant max operation on unsigned integer type

Location: https://github.com/glamsystems/glam/blob/bafcf0eab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/state/price/drift.rs#L116-L117

>_drift.rs

RUST

```
114:         }  
115:  
116:         let oracle_price = oracle_price.max(0);  
117:  
118:         let base_asset_value = (self.base_asset_amount as i128)  
119:             .checked_mul(oracle_price as i128)?
```

Description:

`get_unrealized_pnl()` calls `oracle_price.max(0)` even though `oracle_price` is a `u64`. Unsigned values are already non-negative, so the comparison has no effect.

Potential Benefit:

Reduces a redundant operation and saves compute units.

Recommendation:

Remove the unnecessary `max` call:

```
- let oracle_price = oracle_price.max(0);
```

DIFF

E04: Unused vec parameter in process_markets() adds unnecessary complexity

Location: https://github.com/glamsystems/glam/blob/bafcf9ab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_mint/src/instructions/price_drift_users.rs#L49-L62

> _price_drift_users.rs

RUST

```
47:
48:     // Index markets by IDs
49:     let mut spot_markets: Vec<(u16, SpotMarketSummary)> = Vec::new();
50:     let mut perp_markets: Vec<(u16, PerpMarketSummary)> = Vec::new();
51:     let mut spot_markets_map: BTreeMap<u16, &SpotMarketSummary> = BTreeMap::new();
52:     let mut perp_markets_map: BTreeMap<u16, &PerpMarketSummary> = BTreeMap::new();
53:     process_markets:::<SpotMarketSummary>(
54:         markets_and_oracles,
55:         &mut spot_markets,
56:         &mut spot_markets_map,
57:     )?;
58:     process_markets:::<PerpMarketSummary>(
59:         markets_and_oracles,
60:         &mut perp_markets,
61:         &mut perp_markets_map,
62:     )?;
63:
64:     let now_slot = Clock::get()?.slot;
```

Description:

`process_markets()` in `glam_protocol/src/state/summary.rs` accepts `vec: &'a mut Vec<(u16, T)>` even though the caller never uses the populated vector. The function fills the vector with market summaries, immediately iterates over it to populate the `map` parameter, and callers only read `map`.

Potential Benefit:

Clarifies intent and reduces boilerplate for callers.

Recommendation:

Remove the unused `vec` parameter and populate the map directly.

E05: Protocol fee configuration in GlobalConfig is not used during mint initialization

Location: https://github.com/glamsystems/glam/blob/bafcf704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_mint/src/instructions/initialize_mint.rs#L43-L44

> **_initialize_mint.rs**

RUST

```
41:         let fee_structure = FeeStructure {
42:             protocol: ProtocolFees {
43:                 base_fee_bps: 1,
44:                 flow_fee_bps: 2000,
45:             },
46:             ..mint_model.fee_structure.clone().unwrap_or_default()
```

Description:

GlobalConfig in `glam_config/src/state.rs` defines **base_fee_bps** and **flow_fee_bps**, but **create_initial_mint_params()** hardcodes **base_fee_bps: 1** and **flow_fee_bps: 2000**. As a result, new vaults ignore values configured through **update_protocol_fees()**.

Potential Benefit:

Enables centralized fee management and ensures consistent configuration across vaults.

Recommendation:

Pass the **GlobalConfig** account into **create_initial_mint_params()** and read the fee values from it:

```
base_fee_bps: global_config.base_fee_bps,
flow_fee_bps: global_config.flow_fee_bps,
```

RUST

E06: Tokenized vaults can transfer SOL to any allowed address

Location: https://github.com/glamsystems/glam/blob/bafcfcab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/instructions/system.rs#L80-L93

```
>_ system.rs RUST
78:         )?;
79:     }
80:     _ => {
81:         require!(
82:             allowlist.contains(ctx.accounts.to.key),
83:             GlamError::UnauthorizedSigner
84:         );
85:         check_access_any(
86:             glam_state,
87:             &ctx.accounts.glam_signer.key,
88:             ctx.program_id,
89:             SupportedProtocols::System.bits(),
90:             SystemPermissions::Transfer.bits(),
91:         )?;
92:     }
93: }
94: anchor_lang::system_program::transfer(
95:     CpiContext::new_with_signer(
```

Description:

GLAM_PROTOCOL.system_transfer enforces the **TokenizedVault** restriction when sending SOL to a WSOL token account, but skips the same check when transferring to a system account. Tokenized vaults can therefore send SOL to arbitrary system accounts and bypass intended controls.

Potential Benefit:

Keeps SOL transfers consistent with WSOL handling and prevents unauthorized outflows.

Recommendation:

Add the same account-type check for direct SOL transfers to system accounts.

E07: Kamino/Drift deposit and withdraw operations skip markets allowlist checks

Location: https://github.com/glamsystems/glam/blob/bafcf9ab704126408fdd14c156a26901bdfb9dcb/anchor/programs/ext_drift/src/instructions/vaults_deposit.rs#L17-L29

> _vaults_deposit.rs

RUST

```
15:     amount: u64,
16: ) -> Result<> {
17:     check_integration_and_signer_access_any(
18:         &ctx.accounts.glam_state,
19:         &ctx.accounts.glam_signer.key(),
20:         ctx.program_id,
21:         SupportedProtocols::DriftVaults.bits(),
22:         DriftVaultsPermissions::Deposit.bits(),
23:     )?;
24:     let ix = drift_vaults::instruction::Deposit { _amount: amount };
25:     let mut data = Vec::with_capacity(256);
26:     data.extend_from_slice(drift_vaults::instruction::Deposit::DISCRIMINATOR);
27:     AnchorSerialize::serialize(&ix, &mut data)?;
28:
29:     let mut remaining_accounts = vec![
30:         ctx.accounts.vault.to_account_info(),
31:         ctx.accounts.vault_depositor.to_account_info(),
```

Description:

Both the Kamino and Drift integrations skip checking **markets_allowlist/vaults_allowlist** during deposit and withdrawal flows. As a result, users can continue depositing into markets or vaults that have been removed from the whitelist.

Potential Benefit:

Clarifies the expected behavior once a market is delisted: withdrawals remain allowed, but new deposits are blocked.

Recommendation:

Enforce the allowlist check for deposits (while keeping withdrawals unrestricted) so capital cannot flow into markets or vaults that are no longer approved.

E08: Strengthen Drift borrow guard

Location: https://github.com/glamsystems/glam/blob/bafcf704126408fdd14c156a26901bdfb9dcb/anchor/programs/ext_drift/src/instructions/withdraw.rs#L64-L77

```
> _withdraw.rs RUST
62:     reduce_only: bool,
63: ) -> Result<> {
64:     // Iterate through remaining accounts to find the spot market by market index
65:     let mut spot_market: Option<SpotMarketSummary> = None;
66:     for account in ctx.remaining_accounts {
67:         let data: &[u8] = &account.try_borrow_data()?;
68:         let discm = &data[0..8];
69:         if discm == SpotMarketSummary::discriminator() {
70:             let market = SpotMarketSummary::deserialize(&account.try_borrow_data())?;
71:             if market.market_index == market_index {
72:                 spot_market = Some(market);
73:                 break;
74:             }
75:         }
76:     }
77:     let spot_market = spot_market.ok_or(GlamError::MissingAccount)?;
78:
79:     // Check if user has an existing position with the market_index
```

Description:

The instruction trusts accounts provided in `.remaining_accounts` and never verifies that they are owned by `drift::id()`. Although Drift will eventually reject invalid accounts, the top-level instruction should fail fast.

Potential Benefit:

Reduces wasted compute units and surfaces errors earlier.

Recommendation:

Require the actual Drift spot-market account as a named account, verify its owner and discriminator, deserialize it from that verified data, and stop relying on untrusted `remaining_accounts`.

E09: Incorrect validation of stake pool account

Location: https://github.com/glamsystems/glam/blob/bafcf9ab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/instructions/jupiter_swap.rs#L124-L131

```
>_jupiter_swap.rs RUST
122:
123:     // Pool mint should match the provided mint
124:     let data = match stake_pool_account.try_borrow_data() {
125:         Ok(data) => data,
126:         Err(_) => return false,
127:     };
128:     let pool_mint = match Pubkey::try_from(&data[POOL_MINT_OFFSET..POOL_MINT_OFFSET +
129:     32]) {
130:         Ok(mint) => mint,
131:         Err(_) => return false,
132:     };
133:     pool_mint == *mint // return true if pool mint matches the provided mint
```

Description:

The validation logic only checks that the account owner is one of the three official stake-pool program IDs. Other account types within the same program (for example **ValidatorList**) share that owner, so the check can accept the wrong account type.

Potential Benefit:

Verifying the concrete account type would surface misconfigurations early.

Recommendation:

Deserialize the account into the **StakePool** type or, at minimum, verify the expected data length before proceeding.

E10: Eliminate dead code around underflow check

Location: https://github.com/glamsystems/glam/blob/bafcf9ab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/state/price/kamino.rs#L331-L333

```
>_kamino.rs RUST
329:     // deduct fees from current aum
330:     vault_aum = vault_aum - mgmt_charge - perf_charge;
331:     if vault_aum < 0 {
332:         panic!("kamino vault aum is negative after deducting fees"); // should not hap
           pen but just in case
333:     }
334:
335:     // price vault shares:
```

Description:

The current **if** statement guards against negatives even though the subtraction already panics whenever the result would be negative (these are unsigned integers). The guard is therefore dead code.

Potential Benefit:

Removes redundant logic and keeps failure semantics explicit.

Recommendation:

Compare **vault_aum** against **mgmt_charge + perf_charge** instead of subtracting directly.

E11: Fail gracefully when trying to divide by zero by mistake

Location: https://github.com/glamsystems/glam/blob/bafcf9eb704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/state/price/kamino.rs#L342

> **kamino.rs**

RUST

```
340:         .cast::<u128>()?
341:         .safe_mul(vault_aum.to_floor())?
342:         .safe_div(vault_state_summary.shares_issued.cast())?);
343:
344:     let (lamports, scaled_usd) = price_asset_amount(
```

Description:

`vault_state_summary.shares_issued` should always be greater than zero when pricing makes sense. A division by zero currently triggers a panic without context.

Potential Benefit:

Produces a clear error when an unexpected zero slips through.

Recommendation:

Add a sanity check that requires **`vault_state_summary.shares_issued > 0`** before dividing.

E12: Underwater position stalls pricing and AUM calculation due to problematic cast

Location: https://github.com/glamsystems/glam/blob/bafcf704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_mint/src/instructions/price_drift_vault_depositors.rs#L171

> **_price_drift_vault_depositors.rs**

RUST

```
169:         let vault_aum_scaled_usd = total_spot_value
170:         .safe_add(total_unrealized_pnl)?
171:         .cast::()?
172:         .safe_add(manager_borrowed_scaled_usd)?;
173:
```

Description:

A cast in this function forces values into unsigned types, causing pricing to fail when a position is underwater (unrealized losses exceed collateral). Given Drift's high leverage, underwater positions can occur frequently and block pricing and AUM updates for the entire vault.

Potential Benefit:

Allows pricing to succeed even when individual positions are underwater.

Recommendation:

Keep the value as a signed integer; **PriceProtocol** fields already accept signed representations.

E13: Vault fees documentation concern around burning

Location: https://github.com/glamsystems/glam/blob/bafcf9eab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_mint/src/state/request_queue.rs#L60-L64

```
>_request_queue.rs RUST
58:         let shares_after_fees = shares_before_fees.safe_sub(total_fee_shares)?;
59:
60:         let shares_to_mint = shares_after_fees.safe_add(
61:             pa_shares_manager_fee
62:                 .safe_div(PRECISION_ADJUSTMENT.cast())?
63:                 .cast()?,
64:         )?;
65:
66:         Ok((
```

Description:

pa_shares_vault_fee is supposed to be burned. The code optimizes by skipping the mint-and-burn cycle, which saves compute but creates confusing blockchain traces because no burn appears on explorers.

Potential Benefit:

Aligns on-chain traces with documentation and user expectations.

Recommendation:

Mint and burn **pa_shares_vault_fee** so the burn is visible on-chain.

E14: Move `crystallize_fees_and_calc_new_shares` before fulfilling orders

Location: https://github.com/glamsystems/glam/blob/bafcf0eab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_mint/src/instructions/subscribe.rs#L191-L201

> `_subscribe.rs`

RUST

```
189:     }
190:
191:     // Crystallize fees and mint fee shares to escrow
192:     let misc_fee_shares = crystallize_fees_and_calc_new_shares(
193:         fee_structure,
194:         fee_params,
195:         claimable_fees,
196:         aum,
197:         base_asset_decimals,
198:         Clock::get()?.unix_timestamp,
199:         shares_supply,
200:         shares_mint_decimals,
201:     )?;
202:     let manager_subscription_fee_shares = shares_to_user_and_escrow.safe_sub(shares_to
    _user)?;
203:     let escrow_new_fee_shares = manager_subscription_fee_shares.safe_add(misc_fee_shar
    es)?;
```

Description:

Instructions such as `subscribe` and `fulfill` call `crystallize_fees_and_calc_new_shares` after fulfilling requests, meaning the crystallization uses data that includes the just-processed liquidity changes. Crystallizing with pre-action data is the intended behavior, but the current ordering obscures that intent and increases the risk of future mistakes.

Potential Benefit:

Makes the intended sequencing explicit and guards against regressions.

Recommendation:

Invoke `crystallize_fees_and_calc_new_shares` before mutating state so crystallization always works off the prior snapshot.

E15: Ensure `shares_decimals >= base_asset_decimals`

Location: https://github.com/glamsystems/glam/blob/bafcfceab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/state/flow.rs#L194

```
>_flow.rs RUST
192:         // if aum decimals is less than 9, we need to scale it up
193:         let nav_adjusted = aum
194:         .safe_mul(10_i128.pow((shares_decimals - aum_decimals).try_into().unwrap()
195:         ))?
196:         .safe_mul(PRECISION_ADJUSTMENT)?
197:         .safe_div(shares_supply.into())?;
```

Description:

The code assumes `shares_decimals >= base_asset_decimals` but never enforces it. A misconfiguration would break the implied scaling.

Potential Benefit:

Prevents subtle misconfigurations.

Recommendation:

Add a guard that ensures `shares_decimals >= base_asset_decimals`.

E16: Add extra requirements before closing the GlamProtocol

Location: https://github.com/glamsystems/glam/blob/bafcf9ab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/instructions/state.rs#L142-L146

```
> _state.rs RUST
140:
141: #[glam_macros::glam_vault_signer_seeds]
142: pub fn close_state_handler(ctx: Context<CloseState>) -> Result<()> {
143:     require!(
144:         ctx.accounts.glam_state.mint == Pubkey::default(),
145:         GlamError::CannotCloseState
146:     );
147:
148:     // Transfer vault lamports to signer
```

Description:

close_state_handler allows instant closure of non-tokenized vaults (**mint == Pubkey::default()**) after only checking that the state is enabled and owned. Accidental or malicious closure can lead to data loss.

Potential Benefit:

Protects against mistaken or hostile closures.

Recommendation:

Add additional validation before closing, such as requiring the state to be disabled, assets and **external_positions** to be empty, AUM to be zero, or introducing a two-step timelock.

E17: `cpi_proxy` could validate `cpi_program` the same way integration programs do

Location: https://github.com/glamsystems/glam/blob/bafcf704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/instructions/cpi_proxy.rs#L143-L144

> `cpi_proxy.rs`

RUST

```
141:     pub glam_signer: Signer<'info>,
142:
143:     /// CHECK: trust integration programs to pass correct cpi_program
144:     pub cpi_program: UncheckedAccount<'info>,
145:     pub integration_authority: Signer<'info>,
146:     pub system_program: Program<'info, System>,
```

Description:

Integration programs validate the `cpi_program` before invoking `cpi_proxy`, but `cpi_proxy` itself performs no such check even though the expected program is known.

Potential Benefit:

Strengthens the vault PDA's defenses against misrouted CPI calls.

Recommendation:

Apply the same program-ID validation inside `cpi_proxy`.

E18: Add threshold support against full amount delegate access

Location: https://github.com/glamsystems/glam/blob/bafcf9ab704126408fdd14c156a26901bdfb9dcb/anchor/programs/ext_kamino/src/instructions/farms_stake.rs#L14-L26

> **farms_stake.rs**

RUST

```
12:
13: #[glam_macros::integration_authority_signer_seeds]
14: pub fn farms_stake<'c: 'info, 'info>(
15:     ctx: Context<'_, '_, 'c, 'info, FarmsStake<'info>>,
16:     amount: u64,
17: ) -> Result<()> {
18:     check_integration_and_signer_access_any(
19:         &ctx.accounts.glam_state,
20:         &ctx.accounts.glam_signer.key(),
21:         ctx.program_id,
22:         SupportedProtocols::Farms.bits(),
23:         FarmsPermissions::Stake.bits(),
24:     )?;
25:
26:     let ix = kamino_farm::instruction::Stake { _amount: amount };
27:     let mut data = Vec::with_capacity(256);
28:     data.extend_from_slice(kamino_farm::instruction::Stake::DISCRIMINATOR);
```

Description:

Delegates currently receive all-or-nothing access once granted permissions for an integration and protocol. For example, a delegate with **FarmsPermissions::STAKE** can stake 100% of vault funds in a single instruction. No thresholds or rate limits constrain how much capital a delegate can move.

Potential Benefit:

Reduces the blast radius of compromised credentials or malicious delegates.

Recommendation:

Extend the permission system with quantitative thresholds (for example, maximum amount per action or per epoch) so delegates cannot exercise full control by default.

E19: Enforce Checks on `extra_params` for Each Action

Location: https://github.com/glamsystems/glam/blob/bafcfcab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/instructions/cpi_proxy.rs#L70-L72

```
>_ cpi_proxy.rs RUST
68:     )?;
69:
70:     // Handle extra params
71:     for extra_params in vec_extra_params {
72:         match extra_params.action_type {
73:             ActionType::AddExternalAccount => ctx
74:                 .accounts
```

Description:

The `extra_params` vector passed to `cpi_proxy_handler` is never reconciled against the delegate's granted permissions. Any approved delegate, even one restricted to `FarmsPermissions::HARVEST_REWARD`, can submit `ExtraParams` entries that reference arbitrary `ActionType` values. That enables low-privilege delegates to trigger operations such as adding or removing external accounts, bypassing intended access controls.

Potential Benefit:

Integration programs currently supply `extra_params`, but leaving the handler unchecked is a defensive gap. Future features, misconfigurations, or compromised integrations could abuse this flexibility to escalate privileges.

Recommendation:

Add permission checks inside `cpi_proxy_handler` so every `ExtraParams::action_type` is validated against the delegate's explicit permissions before the action executes.

E20: Lack of validation on destination accounts allows fund diversion by delegator

Location: https://github.com/glamsystems/glam/blob/bafcfcab704126408fdd14c156a26901bdfb9dcb/anchor/programs/ext_drift/src/instructions/withdraw.rs#L176-L178

```
> _withdraw.rs RUST
174:    /// CHECK: should be validated by target program
175:    pub drift_signer: UncheckedAccount<'info>,
176:    /// CHECK: should be validated by target program
177:    #[account(mut)]
178:    pub user_token_account: UncheckedAccount<'info>,
179:    /// CHECK: should be validated by target program
180:    pub token_program: UncheckedAccount<'info>,
```

Description:

The instruction assumes Drift validates the destination account during withdrawals. However, Drift's [withdraw instruction](#) performs no such check, unlike the [deposit instruction](#), which does enforce it.

Potential Benefit:

Strengthening destination account validation keeps funds under protocol control, preventing accidental or malicious diversions and preserving accurate accounting.

Recommendation:

Enforce strict validation on every destination account (even when the underlying protocol performs its own checks) to ensure it always resolves to the **glam_vault** ATA owner.

E21: Unsynced WSOL ATAs mispriced in `validate_and_price_token_account`

Location: https://github.com/glamsystems/glam/blob/bafcf9ab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/state/price/vault.rs#L105-L125

```

>_vault.rs
103:     require_keys_eq!(ata.mint, mint_info.key(), GlamError::InvalidAuthority); // should be redundant but just in case
104:
105:     // Early-out case #2: ata.amount == 0
106:     if ata.amount == 0 {
107:         return Ok(PricedAssetInfo {
108:             lamports: 0,
109:             scaled_usd: 0,
110:             base_asset: 0,
111:             rent: ata_info.lamports(),
112:             account: mint_info.key(), // track mint of a priced token, not the ata
113:         });
114:     }
115:
116:     // Early-out case #3: WSOL
117:     if ata.mint == WSOL {
118:         return Ok(PricedAssetInfo {
119:             lamports: ata.amount.cast()?,
120:             scaled_usd: 0,
121:             base_asset: 0,
122:             rent: ata_info.lamports().safe_sub(ata.amount)?, // WSOL_rent = lamports - token_amount
123:             account: mint_info.key(), // track mint of a priced token, not the ata
124:         });
125:     }
126:
127:     // Early-out case #4: base asset

```

Description:

`validate_and_price_token_account` short-circuits on `ata.amount == 0` before it checks for wrapped SOL. For an unsynced WSOL account, the SPL header still shows a positive lamport balance and `is_native` reserve, but amount remains zero until `sync_native` runs. In that state the code returns `lamports: 0`, so the vault ignores the true SOL balance. If amount happens to be non-zero while the account is still unsynced, the WSOL branch fires but it returns `lamports: ata.amount` instead of the real value (`lamports - rent_reserve`). Either path understates WSOL holdings whenever the ATA isn't freshly synchronized.

Potential Benefit:

Accurate WSOL token accounting prevents financial miscalculations in vault operations.

Recommendation:

Handle WSOL before the `ata.amount == 0` early-out and compute both lamports and rent from the account header, e.g.:

```

if ata.mint == WSOL {
    let rent_reserve = ata.is_native.and_then(|c| c.into()).unwrap_or(0);
    let lamports_on_account = ata_info.lamports();
    return Ok(PricedAssetInfo {
        lamports: lamports_on_account.safe_sub(rent_reserve)?,

```

... continued

RUST

```
scaled_usd: 0,  
base_asset: 0,  
rent: rent_reserve,  
account: mint_info.key(),  
});  
}
```

This uses the native reserve regardless of whether **sync_native** has been called, ensuring WSOL balances are priced correctly in both synced and unsynced states.

E22: Missing error handling for optional position valuation returns

Location: https://github.com/glamsystems/glam/blob/bafcf0eab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/state/price/drift.rs#L191-L202

```
> _drift.rs RUST
189:         get_oracle_price(&oracle_source, oracle, DRIFT_ORACLE_MAX_AGE_SECONDS, now
            _ts)?;
190:
191:         if let Some((value, _)) =
192:             spot_position.get_balance_value_and_token_amount(oracle_price_data.price,
                spot_market)
193:         {
194:             match spot_position.balance_type {
195:                 SpotBalanceType::Deposit => {
196:                     total_spot_value = total_spot_value.safe_add(value.cast())?;
197:                 }
198:                 SpotBalanceType::Borrow => {
199:                     total_spot_value = total_spot_value.safe_sub(value.cast())?;
200:                 }
201:             }
202:         }
203:     }
204:
```

Description:

The `drift_user_spot_and_perp_values()` function calculates total spot value and unrealized PnL for Drift positions. When `get_balance_value_and_token_amount()` returns **None** for spot positions or `get_unrealized_pnl()` returns **None** for perp positions, the function silently skips them via `if let Some(...)` guards.

Those helpers return **None** when arithmetic fails (overflow, underflow, division errors). Ignoring the failures drops affected positions from the totals, producing materially incorrect **total_spot_value** and **total_unrealized_pnl**, which in turn leads to mispriced vault shares or inaccurate collateral checks.

The same pattern appears [here](#).

Potential Benefit:

Ensures accurate position valuations by preventing silent calculation failures that compromise financial data integrity.

Recommendation:

Replace optional handling with explicit error returns.

E23: Inconsistent max age seconds configuration for aggregator oracles

Location: https://github.com/glamsystems/glam/blob/bafcf7eab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/state/price/oracle.rs#L595-L599

```
>_oracle.rs RUST
593:         // Max age seconds defined in local config is preferred
594:         // If not found, use the value from global config
595:         let max_age_seconds = oracle_configs
596:             .and_then(|oracle_configs| {
597:                 oracle_configs.get_max_age_seconds_by_index(asset_meta_idx as u16)
598:             })
599:             .unwrap_or(asset metas[asset_meta_idx].max_age_seconds);
600:
601:         return Ok((
```

Description:

The **validate_oracle()** function in @glam_protocol/src/state/price/oracle.rs handles **max_age_seconds** configuration inconsistently between direct oracles and aggregator oracles.

When a direct oracle is matched, the function properly prioritizes local configuration from **oracle_configs** before falling back to the global configuration:

```
RUST
let max_age_seconds = oracle_configs.and_then(|oracle_configs| {
    oracle_configs.get_max_age_seconds_by_index(asset_meta_idx as u16)
})
.unwrap_or(asset metas[asset_meta_idx].max_age_seconds);
```

However, when using aggregator oracles accessed through the Scope program's **OraclePrices** and **OracleMappings**, the function returns the global configuration without checking for local overrides:

```
RUST
Ok((
    asset metas[asset_meta_idx].oracle_source,
    asset metas[asset_meta_idx].decimals,
    asset metas[asset_meta_idx].max_age_seconds,
))
```

This prevents vault owners from customizing oracle freshness requirements on a per-fund basis when using aggregated price feeds, forcing all funds to share the same staleness tolerance regardless of their individual vault risk preferences. The behaviour is a configuration gap rather than a security exploit.

Potential Benefit:

Allows each vault to enforce its preferred staleness threshold even when using aggregated feeds, improving risk management flexibility without protocol-wide changes.

Recommendation:

Apply the same local-first configuration logic in the aggregator oracle path.

E24: Output token account must be pre-created for Jupiter swaps

Location: https://github.com/glamsystems/glam/blob/bafcf7eab704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/instructions/jupiter_swap.rs#L264-L270

```
>_jupiter_swap.rs RUST
262:     )]
263:     pub input_vault_ata: Box<InterfaceAccount<'info, TokenAccount>>,
264:     #[account(
265:         mut,
266:         associated_token::mint = output_mint,
267:         associated_token::authority = glam_vault,
268:         associated_token::token_program = output_token_program
269:     )]
270:     pub output_vault_ata: Box<InterfaceAccount<'info, TokenAccount>>,
271:     pub input_mint: Box<InterfaceAccount<'info, Mint>>,
272:     pub output_mint: Box<InterfaceAccount<'info, Mint>>,
```

Description:

In the `jupiter_swap()` instruction, the `output_vault_ata` account constraint requires the associated token account to exist before executing a swap. When a vault swaps to a new output token for the first time, the transaction will revert if the output ATA hasn't been pre-created.

```
RUST
#[account(
    mut,
    associated_token::mint = output_mint,
    associated_token::authority = glam_vault,
    associated_token::token_program = output_token_program
)]
pub output_vault_ata: Box<InterfaceAccount<'info, TokenAccount>>,
```

This creates unnecessary friction for vault managers performing swaps to new assets.

Potential Benefit:

Improved user experience by allowing automatic ATA creation during swaps.

Recommendation:

Add `init_if_needed` constraint to automatically create the output ATA.

E25: Fee structure changes can be applied to pending requests when timelock is disabled

Location: https://github.com/glamsystems/glam/blob/bafcf704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_mint/src/instructions/update_mint_apply_timelock.rs#L34-L40

> **update_mint_apply_timelock.rs**

RUST

```
32:         );
33:         let request_queue = ctx.accounts.request_queue.as_ref().unwrap();
34:         require!(
35:             !request_queue
36:                 .data
37:                 .iter()
38:                 .any(|r| r.is_pending_fulfillment()),
39:             GlamError::CannotApplyChanges
40:         );
41:     }
42:
```

Description:

The protocol validates that no pending fulfillments exist before applying mint parameter updates in **update_mint_apply_timelock()**. This prevents fee structure changes from being retroactively applied to existing subscription and redemption requests:

```
require!(
    !request_queue.data.iter().any(|r| r.is_pending_fulfillment()),
    GlamError::CannotApplyChanges
);
```

RUST

However, when **timelock_duration == 0**, the **update_mint()** function applies changes immediately via **materialize_mint_updates()** without performing this validation. This allows an admin to modify the fee structure and have it immediately apply to all pending requests, potentially charging users fees that differ from what they expected when submitting their requests.

Potential Benefit:

Ensures pending requests are honored under the terms that were active when users submitted them, keeping fee changes predictable even without a timelock.

Recommendation:

Add the same pending fulfillment validation to **update_mint()** when **timelock_duration == 0**.

E26: Max-staleness oracle aggregation

Location: https://github.com/glamsystems/glam/blob/bafcf9eb704126408fdd14c156a26901bdfb9dcb/anchor/programs/glam_protocol/src/state/price/oracle.rs#L349-L354

```
>_oracle.rs RUST
347:     let data = &price_oracle.try_borrow_mut_data()?;
348:     let (submissions, range, last_update_timestamp) = parse_sb_on_demand(data)?;
349:     let max_landed_at = submissions.iter().map(|s| s.landed_at).max().unwrap_or(0);
350:     let latest_submissions = submissions
351:         .iter()
352:         .filter(|s| s.landed_at == max_landed_at)
353:         .cloned()
354:         .collect::<Vec<_>>();
355:
356:     let average_price = latest_submissions
```

Description:

The current logic filters strictly on **max_landed_at**, so only submissions with the very latest timestamp contribute to the price. That's acceptable because the averaged result is still checked against the confidence interval, but it needlessly discards other fresh data. Adding a **max_staleness** window, similar to [Switchboard's approach](#) would let the aggregator pull in multiple recent submissions, preserving the confidence checks while giving the price feed more depth and resilience.

Potential Benefit:

Broader eligibility for fresh submissions increases price stability and reduces variance when one oracle lands slightly later than peers.

Recommendation:

Introduce a configurable **max_staleness** parameter and aggregate every submission whose **landed_at** falls within that window, mirroring the Switchboard SDK pattern so additional timely data points participate in the averaging step.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification

- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.

Appendix A: LiteSVM Queue Stress Harness

Findings at a Glance

- With the initial queue implementation, a 200_000 compute-unit cap limited **Fulfill** to roughly 100 pending requests before the transaction exhausted its budget.
- After the queue rework (zero-copy layout + bounded iteration), the same harness processed approximately 5_000 requests under the identical 200_000 CU ceiling an improvement of about 50x.
- The production cap of 1_000 requests remains deliberately conservative, and we retained an optional per-call limit inside **Fulfill** as a safety net should future changes threaten that margin.

Patch Context

Apply `queue_litesvm_test.patch` relative to commit `bafcfeab704126408fdd14c156a26901bdfb9dcb`; the file paths listed below match that revision.

Purpose

`anchor/programs/glam_mint/tests/litesvm_smoke.rs::fulfill_large_queue_exceeds_compute_budget` exercises the **Fulfill** instruction against synthetic request queues inside a LiteSVM environment. The harness reproduces the compute blow-up we observed on large queues and provides an easy toggle to confirm that the remediation keeps compute usage below the target unit ceiling (1.4M or 200_000).

Test Setup

I. **Build the on-chain programs** so the `.so` artifacts are available under `anchor/target/deploy` (the harness loads them at runtime). Do this before applying the LiteSVM patch, because the additional dependencies introduced here are not compatible with the stock `anchor build` flow. Apply the patch next.

II. **Execute the LiteSVM test** from `anchor/`:

```
cargo test -p glam_mint fulfill_large_queue_exceeds_compute_budget --test litesvm_smoke BASH
```

The test fabricates all required accounts (vault state, mints, ATAs, request queue) and submits back-to-back **Fulfill** calls so you can observe how compute usage scales as the queue length grows.

Tuning the Boundaries

These constants live near the middle of the test file. Tweaking them is the fastest way to explore different queue sizes and compute budgets:

```
const SMALL_QUEUE_LEN: usize = 64;  
const LARGE_QUEUE_LEN: usize = 100;  
const COMPUTE_LIMIT: u32 = 1_400_000;
```

RUST

- Increase either queue length to simulate heavier backlogs, or dial the compute limit up/down to match the budget you expect on mainnet.
- After each adjustment, re-run **cargo test ...** to capture the new compute profile, fixtures are deterministic, so before/after comparisons are trivial.

Keep this harness alongside other regression tests, but wire it into ad-hoc or nightly job suites rather than the default **anchor test** path. LiteSVM's extra dependencies can surprise CI pipelines that expect the stock Anchor toolchain. It remains the quickest way to validate future changes to **RequestQueue::fulfill** or related data structures once you account for that wrinkle.

Appendix B: AFL Fuzzing for Queue Subscriptions

Patch Context

Apply `fuzzing_starter.patch` relative to commit `bafcf9ab704126408fdd14c156a26901bdfb9dcb`; all paths and instructions below match that revision.

Findings at a Glance

- We built a skeleton AFL harness for `glam_mint` (initially wired to just `subscribe` but can be extended for all queue operations) so the team can plug fuzzing into their workflow without starting from scratch.
- The harness runs directly against the Rust binary (`target/debug/glam-fuzz`) instead of the compiled `.so`, letting libAFL's coverage instrumentation work without extra glue code.
- Invariants guard every iteration: deviations in total shares, claimable fees, AUM, or NAV terminate the fuzz case, so regressions surface as hard failures.

Build & Run Workflow

I. Compile the fuzz target

```
cd anchor
cargo afl build -p glam-fuzz
```

BASH

This produces `target/debug/glam-fuzz` with AFL instrumentation enabled.

II. Prepare the initial corpus

```
mkdir -p seeds
dd if=/dev/urandom of=seeds/seed0 bs=1 count=100
```

BASH

This single random seed lets AFL start mutating immediately; replace it with protocol-aware seeds as they become available.

III. Launch the fuzzer

The AFL harness defined in `anchor/fuzz/glam-fuzz/src/main.rs` executes the following entrypoint, which wires AFL's `fuzz!` macro into `drive_case`:

```
fn main() {
    setup_with_default("solana_program_test=error,solana_rbpf=error");

    fuzz!(|data: &[u8]| {
        runtime().block_on(drive_case(data));
    });
}
```

RUST

```
AFL_I_DONT_CARE_ABOUT_MISSING_CRASHES=1 \  
AFL_NO_UI=1 \  
AFL_DEBUG_CHILD=1 \  
cargo afl fuzz -i seeds -o findings -- target/debug/glam-fuzz
```

BASH

- **AFL_I_DONT_CARE_ABOUT_MISSING_CRASHES=1** suppresses macOS-specific warnings; Linux runners typically do not need it.
- **AFL_NO_UI=1** disables the full-screen UI so stdout shows harness logs; omit it when you want the standard AFL dashboard.
- **AFL_DEBUG_CHILD=1** pipes debug output from the target; remove it for faster, quieter runs.

IV. Quick sanity check while iterating

Instead of rebuilding and spinning up AFL for every tweak, run the built-in smoke test:

```
#[cfg(test)]  
mod tests {  
    use super::*;  
    use rand::Rng;  
  
    #[test]  
    fn smoke_drive_case() {  
        let mut rng = rand::thread_rng();  
        let mut data = [0u8; 100 * 100 * 100];  
        rng.fill(&mut data);  
  
        runtime().block_on(drive_case(&data));  
    }  
}
```

RUST

This feeds random data into **drive_case** and ensures new assertions or invariants compile and execute without AFL in the loop.

Harness Overview

- **Target:** **anchor/fuzz/glam-fuzz/src/main.rs** feeds mutated byte slices into the on-chain queue logic, constructing subscription instructions inside the process.
- **Fixtures:** Deterministic builders hydrate vault state, escrow ATAs and fee parameters so each iteration starts from a known baseline.
- **Mutation strategy:** libAFL mutates subscription amounts to trigger accounting edge cases.
- **Corpus format:** Seeds live under **seeds/** as TOML snippets, which keeps triage human-friendly and speeds up manual reproduction.

Invariants Enforced

To tolerate rounding that favors the vault while still catching regressions, each check verifies the computed value does not exceed the theoretical maximum:

- Total shares issued by **glam_mint** must be equal the sum of escrow-held shares plus user-held shares.
- AUM must equal the sum of the **amount** fields recorded in **priced_protocols**.
- NAV must equal the AUM divided by total share supply using scaled integers.
- Protocol flow fees are **manager_fee_shares * protocol_fee_bps / 10_000**.
- Claimable fee shares must never fall below the difference between total escrow shares and fulfilled user shares (values above that bound are acceptable because **claim_fees** clamps payouts with **cmp::min**, which also caps the user-facing and protocol-facing share transfers calculated below).
- **Subscription flow:**
 - Shares before manager fees are computed as **user_deposit * (1 - vault_fee_bps / 10_000) / nav**.
 - Shares minted to the user equal the previous result multiplied by **(1 - manager_fee_bps / 10_000)**.

If any check fails, the harness treats the iteration as a crash; AFL minimizes the input and writes it to **findings/** for reproduction.

Extending the Harness

- **Broaden coverage:** Add wrappers for **queued_redeem**, **cancel**, and **fulfill** in **main.rs**, then reuse the same invariant helpers to cover the rest of the queue pipeline.
- **Seed curation:** After significant queue or fee changes, regenerate seeds by running AFL briefly and promoting interesting cases from **findings/queue** back into **seeds/**.
- **Automation:** Schedule nightly fuzz jobs (Linux runners preferred) so the corpus keeps growing alongside the codebase.

By investing in this AFL workflow, the team gets a repeatable, coverage-guided stress test for the subscription queue-complementing the LiteSVM harness and providing early warning on subtle accounting regressions.

Appendix C: Overpricing of external positions POC

```

use std::{path::PathBuf, str::FromStr};

use anchor_lang::{AccountDeserialize, AccountSerialize, Discriminator, InstructionData};
use anchor_spl::associated_token::get_associated_token_address_with_program_id;
use anyhow::{anyhow, Context, Result};
use base64::engine::general_purpose::STANDARD as BASE64;
use base64::Engine;
use ext_kamino::SupportedProtocols as ExtKaminoSupportedProtocols;
use glam_common::constants::{SEED_GLOBAL_CONFIG, SEED_INTEGRATION_AUTHORITY, SEED_VAULT, WSOL};
use glam_config::state::{
    AssetMeta as ConfigAssetMeta, GlobalConfig, OracleSource as ConfigOracleSource,
};
use glam_mint::state::aum::GlamVaultAum;
use glam_protocol::state::{
    accounts::{AccountType, CreatedModel, StateAccount},
    acl::IntegrationAcl,
    assets::PricedProtocol,
    summary::{KaminoVaultSummary, Summary},
};
use litesvm::LiteSVM;
use serde::Deserialize;
use solana_program::{program_option::COption, program_pack::Pack};
use solana_sdk::{
    account::Account as SolanaAccount,
    instruction::{AccountMeta, Instruction},
    pubkey::Pubkey,
    rent::Rent,
    signature::{Keypair, Signer},
    system_program,
    transaction::Transaction,
};
use spl_token::state::{Account as SplTokenAccount, AccountState};

const SOL_ORACLE_PATH: &str =
    "../../fixtures/accounts/drift/3m6i4RFWEDw2Ft4tFHPJtYgmpPe21k56M3FHeWYrgGBz.json";
const USDC_ORACLE_PATH: &str =
    "../../fixtures/accounts/drift/9VCioxmni2gDLv11qufWzT3RDERhQE4iY5Gf7NTfYyAV.json";
const SHARES_MINT_PATH: &str = "../../fixtures/accounts/kamino/kVUSDC_shares_mint.json";
const VAULT_STATE_PATH: &str = "../../fixtures/accounts/kamino/kVUSDC_vault_state.json";
const RESERVE_PATHS: [&str; 3] = [
    "../../fixtures/accounts/kamino/reserve_usdc_jlp_market.json",
    "../../fixtures/accounts/kamino/reserve_usdc_main_market.json",
    "../../fixtures/accounts/kamino/reserve_usdc_maple_market.json",
];

#[derive(Deserialize)]
struct FixtureAccount {
    pub pubkey: String,
    pub account: InnerAccount,
}

#[derive(Deserialize)]
struct InnerAccount {
    pub lamports: u64,

```

RUST

... continued

RUST

```

pub data: [String; 2],
pub owner: String,
pub executable: bool,
#[serde(rename = "rentEpoch")]
pub rent_epoch: u64,
}

#[test]
fn price_kamino_vault_shares_counts_duplicate_accounts() -> Result<()> {
    let Some(glam_mint_program) = load_program_artifact("glam_mint.so") else {
        eprintln!("skipping duplicate-price test: build glam_mint.so first");
        return Ok(());
    };
    let Some(glam_protocol_program) = load_program_artifact("glam_protocol.so") else {
        eprintln!("skipping duplicate-price test: build glam_protocol.so first");
        return Ok(());
    };

    let mut svm = LiteSVM::new();
    svm.add_program(glam_protocol::ID, &glam_protocol_program)
        .map_err(|e| anyhow!("failed to load glam_protocol program: {e:?}"))?;
    svm.add_program(glam_mint::ID, &glam_mint_program)
        .map_err(|e| anyhow!("failed to load glam_mint program: {e:?}"))?;

    // Signer / payer
    let payer = Keypair::new();
    svm.airdrop(&payer.pubkey(), 5_000_000_000)
        .map_err(|e| anyhow!("airdrop failed: {e:?}"))?;

    // Load fixture accounts
    let (sol_oracle_pubkey, sol_oracle_account) = load_fixture_account(SOL_ORACLE_PATH)?;
    svm.set_account(sol_oracle_pubkey, sol_oracle_account)
        .map_err(|e| anyhow!("set sol oracle: {e:?}"))?;

    let (usdc_oracle_pubkey, usdc_oracle_account) = load_fixture_account(USDC_ORACLE_PATH)?;
    svm.set_account(usdc_oracle_pubkey, usdc_oracle_account.clone())
        .map_err(|e| anyhow!("set usdc oracle: {e:?}"))?;

    let (shares_mint_pubkey, shares_mint_account) = load_fixture_account(SHARES_MINT_PATH)?;
    svm.set_account(shares_mint_pubkey, shares_mint_account)
        .map_err(|e| anyhow!("set shares mint: {e:?}"))?;

    let (vault_state_pubkey, vault_state_account) = load_fixture_account(VAULT_STATE_PATH)?;
    let vault_state_data = vault_state_account.data.clone();
    svm.set_account(vault_state_pubkey, vault_state_account)
        .map_err(|e| anyhow!("set vault state: {e:?}"))?;

    let vault_summary = KaminoVaultSummary::deserialize(&vault_state_data)
        .map_err(|e| anyhow!("deserialize vault summary: {e}"))?;

    for path in RESERVE_PATHS {
        let (pubkey, account) = load_fixture_account(path)?;
        svm.set_account(pubkey, account)
            .with_context(|| format!("set reserve {pubkey}"))?;
    }

    // Derive GLAM PDAs
    let glam_state_key = Pubkey::new_unique();

```

... continued

RUST

```

let (glam_vault, _) = Pubkey::find_program_address(
    &[SEED_VAULT.as_bytes(), glam_state_key.as_ref()],
    &glam_protocol::ID,
);
let (glam_config_key, _) =
    Pubkey::find_program_address(&[SEED_GLOBAL_CONFIG.as_bytes()], &glam_config::ID);
let (integration_authority, _) =
    Pubkey::find_program_address(&[SEED_INTEGRATION_AUTHORITY.as_bytes()], &glam_mint::ID)
;

// Create a synthetic ATA for the derived GLAM vault
const SHARE_AMOUNT: u64 = 5_000_000_000;
let share_ata_key = get_associated_token_address_with_program_id(
    &glam_vault,
    &vault_summary.shares_mint,
    &spl_token::ID,
);
let share_ata_state = SplTokenAccount {
    mint: vault_summary.shares_mint,
    owner: glam_vault,
    amount: SHARE_AMOUNT,
    delegate: COption::None,
    state: AccountState::Initialized,
    is_native: COption::None,
    delegated_amount: 0,
    close_authority: COption::None,
};
let mut share_ata_data = vec![0u8; SplTokenAccount::LEN];
SplTokenAccount::pack(share_ata_state, &mut share_ata_data)
    .map_err(|e| anyhow!("pack share ATA: {e}"))?;
svm.set_account(
    share_ata_key,
    SolanaAccount {
        lamports: Rent::default().minimum_balance(SplTokenAccount::LEN),
        data: share_ata_data,
        owner: spl_token::ID,
        executable: false,
        rent_epoch: 0,
    },
)
.map_err(|e| anyhow!("set share ata: {e:?}"))?;

// Configure GLAM state
let glam_state = StateAccount {
    account_type: AccountType::Vault,
    enabled: true,
    vault: glam_vault,
    owner: payer.pubkey(),
    portfolio_manager_name: [0; 32],
    created: CreatedModel {
        key: [0; 8],
        created_by: payer.pubkey(),
        created_at: 0,
    },
    base_asset_mint: vault_summary.deposit_token_mint,
    base_asset_decimals: 6,
    base_asset_token_program: 0,
    name: [0; 32],

```

... continued

RUST

```

        timelock_duration: 0,
        timelock_expires_at: 0,
        mint: Pubkey::default(),
        assets: vec![vault_summary.deposit_token_mint],
        integration_acls: vec![IntegrationAcl {
            integration_program: glam_mint::ID,
            protocols_bitmask: ExtKaminoSupportedProtocols::Vaults.bits(),
            protocol_policies: Vec::new(),
        }],
        delegate_acls: Vec::new(),
        external_positions: vec![share_ata_key],
        priced_protocols: Vec::new(),
        params: vec![vec![], vec![], vec![], vec![]],
    };
    let glam_state_data = serialize_anchor_account(&glam_state, StateAccount::INIT_SIZE);
    svm.set_account(
        glam_state_key,
        SolanaAccount {
            lamports: Rent::default().minimum_balance(StateAccount::INIT_SIZE),
            data: glam_state_data,
            owner: glam_protocol::ID,
            executable: false,
            rent_epoch: 0,
        },
    )
    .map_err(|e| anyhow!("set glam_state: {e:?}"))?;

    // Global config
    let config = GlobalConfig {
        admin: payer.pubkey(),
        fee_authority: payer.pubkey(),
        referrer: payer.pubkey(),
        base_fee_bps: 0,
        flow_fee_bps: 0,
        asset metas: vec![
            ConfigAssetMeta {
                asset: vault_summary.deposit_token_mint,
                decimals: 6,
                oracle: usdc_oracle_pubkey,
                oracle_source: ConfigOracleSource::PythLazerStableCoin,
                max_age_seconds: 0,
                priority: 0,
                padding: [0; 3],
            },
            ConfigAssetMeta {
                asset: WSOL,
                decimals: 9,
                oracle: sol_oracle_pubkey,
                oracle_source: ConfigOracleSource::PythLazer,
                max_age_seconds: 0,
                priority: 0,
                padding: [0; 3],
            },
        ],
    };
    let config_size =
        8 + 32 + 32 + 32 + 2 + 2 + 4 + config.asset_metas.len() * ConfigAssetMeta::SIZE;
    let glam_config_data = serialize_anchor_account(&config, config_size);

```

... continued

RUST

```
svm.set_account(
    glam_config_key,
    SolanaAccount {
        lamports: Rent::default().minimum_balance(config_size),
        data: glam_config_data,
        owner: glam_config::ID,
        executable: false,
        rent_epoch: 0,
    },
)
.map_err(|e| anyhow!("set glam_config: {e:?}"))?;

// Remaining PDAs / system accounts
svm.set_account(
    glam_vault,
    SolanaAccount {
        lamports: Rent::default().minimum_balance(0),
        data: Vec::new(),
        owner: system_program::ID,
        executable: false,
        rent_epoch: 0,
    },
)
.map_err(|e| anyhow!("set glam_vault: {e:?}"))?;
svm.set_account(
    integration_authority,
    SolanaAccount {
        lamports: Rent::default().minimum_balance(0),
        data: Vec::new(),
        owner: glam_mint::ID,
        executable: false,
        rent_epoch: 0,
    },
)
.map_err(|e| anyhow!("set integration authority: {e:?}"))?;

// Instruction account metadata
let base_accounts = vec![
    AccountMeta::new(glam_state_key, false),
    AccountMeta::new_readonly(glam_vault, false),
    AccountMeta::new(payer.pubkey(), true),
    AccountMeta::new_readonly(sol_oracle_pubkey, false),
    AccountMeta::new_readonly(usdc_oracle_pubkey, false),
    AccountMeta::new_readonly(integration_authority, false),
    AccountMeta::new_readonly(glam_config_key, false),
    AccountMeta::new_readonly(glam_protocol::ID, false),
];
let share_chunk = vec![
    AccountMeta::new_readonly(share_ata_key, false),
    AccountMeta::new_readonly(vault_summary.shares_mint, false),
    AccountMeta::new_readonly(vault_state_pubkey, false),
    AccountMeta::new_readonly(usdc_oracle_pubkey, false),
];
let mut reserves_accounts = Vec::new();
for allocation in &vault_summary.allocations {
    let reserve = allocation.reserve;
    svm.get_account(&reserve)
        .ok_or_else(|| anyhow!("missing reserve account {reserve}"))?;
```

... continued

RUST

```
reserves_accounts.push(AccountMeta::new_readonly(reserve, false));
}

// Single pricing pass
let mut unique_remaining = share_chunk.clone();
unique_remaining.extend(reserves_accounts.clone());
run_price_kamino_vault_shares(&mut svm, &payer, &base_accounts, unique_remaining, 1)?;
let single_state = read_glam_state(&svm, glam_state_key)?;
let priced = single_state
    .priced_protocols
    .first()
    .expect("priced protocol missing after first run");
assert_eq!(priced.positions.len(), 1, "expected single position");
assert_eq!(
    priced.positions[0], share_ata_key,
    "unexpected position key after single run"
);
assert!(
    priced.amount > 0,
    "expected positive base asset amount after single run"
);
let single_amount = priced.amount;

// Duplicate pricing pass (same vault listed twice)
let mut duplicate_remaining = Vec::new();
for _ in 0..2 {
    duplicate_remaining.extend(share_chunk.iter().cloned());
}
for _ in 0..2 {
    duplicate_remaining.extend(reserves_accounts.iter().cloned());
}
run_price_kamino_vault_shares(&mut svm, &payer, &base_accounts, duplicate_remaining, 2)?;
let double_state = read_glam_state(&svm, glam_state_key)?;
let priced = double_state
    .priced_protocols
    .first()
    .expect("priced protocol missing after duplicate run");
assert_eq!(
    priced.positions.len(),
    2,
    "duplicate run should record two positions"
);
assert!(
    priced.positions.iter().all(|pk| *pk == share_ata_key),
    "duplicate run stored unexpected position keys"
);
assert_eq!(
    priced.amount,
    single_amount * 2,
    "duplicate accounting should double the priced amount"
);

// Verify AUM only counts the external position once by using the trait logic.
// Provide a zeroed vault-token entry so the AUM check passes vault-token validation.
let mut aum_state = double_state.clone();
let vault_entry = PricedProtocol {
    rent: 0,
    amount: 0,
```

... continued

RUST

```

        decimals: aum_state.base_asset_decimals,
        last_updated_slot: priced.last_updated_slot,
        integration_program: glam_protocol::ID,
        protocol_bitflag: 0,
        positions: Vec::new(),
    };
    aum_state.upsert_priced_protocol(&vault_entry);
    let computed_aum = aum_state.aum(0)?;
    assert_eq!(
        computed_aum,
        priced.amount,
        "AUM should match the deduplicated priced amount"
    );

    Ok(())
}

fn load_program_artifact(name: &str) -> Option<Vec<u8>> {
    let path = PathBuf::from(env!("CARGO_MANIFEST_DIR"))
        .join("../target/deploy")
        .join(name);
    std::fs::read(path).ok()
}

fn load_fixture_account(path: &str) -> Result<(Pubkey, SolanaAccount)> {
    let json_path = PathBuf::from(env!("CARGO_MANIFEST_DIR")).join(path);
    let json = std::fs::read_to_string(&json_path)
        .with_context(|| format!("read fixture {}", json_path.display()))?;
    let fixture: FixtureAccount = serde_json::from_str(&json)
        .with_context(|| format!("parse fixture {}", json_path.display()))?;
    let pubkey = Pubkey::from_str(&fixture.pubkey)
        .map_err(|e| anyhow!("invalid pubkey in {}: {e}", json_path.display()))?;
    let owner = Pubkey::from_str(&fixture.account.owner)
        .map_err(|e| anyhow!("invalid owner in {}: {e}", json_path.display()))?;
    let data = BASE64
        .decode(&fixture.account.data[0])
        .map_err(|e| anyhow!("base64 decode {}: {e}", json_path.display()))?;
    Ok((
        pubkey,
        SolanaAccount {
            lamports: fixture.account.lamports,
            data,
            owner,
            executable: fixture.account.executable,
            rent_epoch: fixture.account.rent_epoch,
        },
    ))
}

fn serialize_anchor_account<T: AccountSerialize + Discriminator>(
    account: &T,
    size_hint: usize,
) -> Vec<u8> {
    let mut data = Vec::new();
    account
        .try_serialize(&mut data)
        .expect("account serialization");
    if data.len() < size_hint {

```

... continued

RUST

```
        data.resize(size_hint, 0);
    }
    data
}

fn run_price_kamino_vault_shares(
    svm: &mut LiteSVM,
    payer: &Keypair,
    base_accounts: &[AccountMeta],
    remaining_accounts: Vec<AccountMeta>,
    num_vaults: u8,
) -> Result<()> {
    let mut accounts = base_accounts.to_vec();
    accounts.extend(remaining_accounts);
    let ix = Instruction {
        program_id: glam_mint::ID,
        accounts,
        data: glam_mint::instruction::PriceKaminoVaultShares { num_vaults }.data(),
    };
    let tx = Transaction::new_signed_with_payer(
        &[ix],
        Some(&payer.pubkey()),
        &[payer],
        svm.latest_blockhash(),
    );
    svm.send_transaction(tx)
        .map(|_| ())
        .map_err(|e| anyhow!("send price transaction: {e:?}"))
}

fn read_glam_state(svm: &LiteSVM, key: Pubkey) -> Result<StateAccount> {
    let account = svm
        .get_account(&key)
        .ok_or_else(|| anyhow!("missing glam_state account {key}"))?;
    let mut data_slice: &[u8] = &account.data;
    StateAccount::try_deserialize(&mut data_slice)
        .map_err(|e| anyhow!("deserialize glam_state: {e}"))
}
```